Module 6 : Feature Engineering for Commodities

Learning Objectives

By the end of this module, you will:

1. Understand the critical importance of temporal discipline in feature engineering
2. Master the `TemporalFeatureEngineer` class for creating time-aware features
3. Build price-based features: returns, volatility, z-scores, and momentum
4. Create fundamental features from inventory, supply-demand, and cost curves
5. Engineer seasonal and interaction features for commodities
6. Apply feature engineering to crude oil price prediction

Why Feature Engineering Matters

"Feature engineering is often the difference between a mediocre model and a great one."

In commodities trading, your features encode economic theories about what drives prices. Good features:

- Capture market microstructure and fundamentals
- Respect temporal causality (no look-ahead bias)
- Are robust to data revisions and gaps
- Generalize across market regimes

In [1]:
```
# Standard imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')

# Set style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.precision', 4)

print("✓ Environment configured")
```

```
✓ Environment configured
```

In [2]:
```
# Import fair_value_toolkit
import sys
sys.path.insert(0, '../../src')
```

```python
from fair_value_toolkit.features import (
    TemporalFeatureEngineer,
    FeatureMetadata,
    create_lagged_features,
    create_rolling_features,
    create_seasonal_features,
    create_commodity_features
)

print("✓ fair_value_toolkit imported")
```

```
✓ fair_value_toolkit imported
```

Part 1 : The Golden Rule - Never Use Future Informa

The Problem with Look-Ahead Bias

Consider this innocent-looking code:

```python
# WRONG - This uses future information!
df['price_zscore'] = (df['price'] - df['price'].mean()) / df['price'].s
```

Why is this wrong?

The `.mean()` and `.std()` are calculated using the **entire dataset**, including future observat

When training on January `2020` data, you're using statistics from December `202`

The Correct Way: Rolling Statistics

In [3]:
```python
# Create synthetic commodity price data
np.random.seed(42)
dates = pd.date_range('2020-01-01', '2023-12-31', freq='D')
n = len(dates)

# Simulate crude oil prices with trend, seasonality, and noise
trend = np.linspace(50, 70, n)
seasonality = 10 * np.sin(2 * np.pi * np.arange(n) / 365.25)
noise = np.random.normal(0, 5, n)
price = trend + seasonality + noise

# Create DataFrame
df = pd.DataFrame({
    'price': price,
    'inventory': 1000 + np.random.normal(0, 100, n).cumsum(),
    'production': 100 + np.random.normal(0, 5, n),
    'consumption': 98 + np.random.normal(0, 5, n),
}, index=dates)

print(f"Created synthetic commodity data: {len(df)} days")
print(f"Date range: {df.index[0].date()} to {df.index[-1].date()}")
df.head()
```

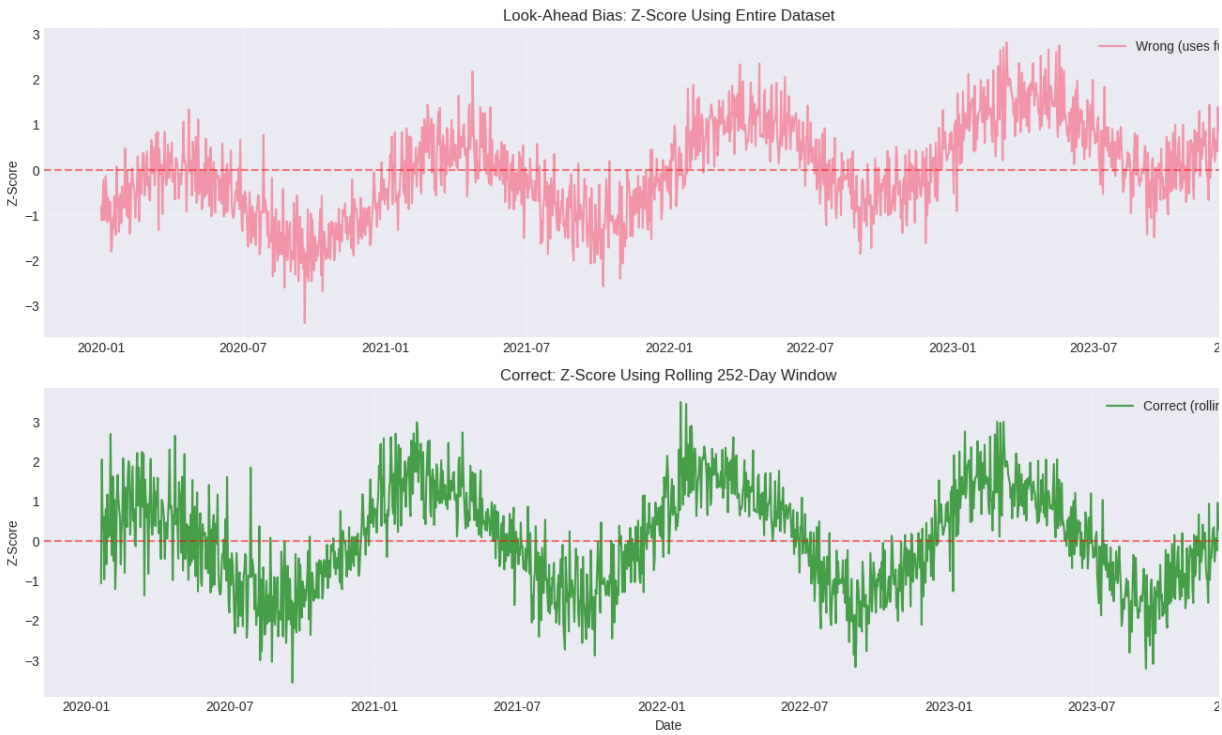
```
Created synthetic commodity data: 1461 days
Date range: 2020-01-01 to 2023-12-31
```

Out[3]:

	price	inventory	production	consum
2020-01-01	52.4836	877.5702	99.0265	98.98
2020-01-02	49.4944	856.6679	99.6791	94.96
2020-01-03	53.6098	771.6158	100.4588	95.97
2020-01-04	58.1721	713.5635	101.2612	94.56
2020-01-05	49.5716	772.4213	99.4189	90.83

In [4]:

```
# Demonstrate look-ahead bias
# WRONG way (uses future data)
df['zscore_wrong'] = (df['price'] - df['price'].mean()) / df['price'].std

# CORRECT way (uses only past data)
window = 252  # 1 year
rolling_mean = df['price'].rolling(window=window, min_periods=20).mean()
rolling_std = df['price'].rolling(window=window, min_periods=20).std()
df['zscore_correct'] = (df['price'] - rolling_mean) / rolling_std

# Compare
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Plot 1: Wrong z-score
axes[0].plot(df.index, df['zscore_wrong'], label='Wrong (uses future data
axes[0].axhline(0, color='red', linestyle='--', alpha=0.5)
axes[0].set_ylabel('Z-Score')
axes[0].set_title('Look-Ahead Bias: Z-Score Using Entire Dataset')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Plot 2: Correct z-score
axes[1].plot(df.index, df['zscore_correct'], label='Correct (rolling wind
axes[1].axhline(0, color='red', linestyle='--', alpha=0.5)
axes[1].set_ylabel('Z-Score')
axes[1].set_xlabel('Date')
axes[1].set_title('Correct: Z-Score Using Rolling 252-Day Window')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nKey Difference:")
print(f"Wrong z-score has mean: {df['zscore_wrong'].mean():.4f} (should b
print(f"Correct z-score early values are NaN until window fills up")
```




```
Key Difference:
Wrong z-score has mean: 0.0000 (should be ~0 everywhere)
Correct z-score early values are NaN until window fills up
```

Key Insight

Notice how the "correct" z-score:

1. Starts with NaN values (until the window fills)
2. Adapts over time as the rolling window moves
3. Never uses future information

The "wrong" z-score appears more stable because it uses information from the entire time series creating an illusion of predictability.

Part 2: The TemporalFeatureEngineer Class

The `TemporalFeatureEngineer` automates temporal feature creation while maintaining stri discipline. It tracks:

- **Feature metadata**: What each feature represents
- **Lookback periods**: How much history is used
- **Transformation logic**: How to apply features consistently

Basic Usage Pattern

In [5]:
```
# Initialize the feature engineer
engineer = TemporalFeatureEngineer(
    target_column='price',
    fill_method='ffill'  # Forward fill missing values
)

# Fit on training data (stores statistics)
train_end = '2022-12-31'
```

```
train_data = df.loc[:train_end].copy()

engineer.fit(train_data)

print("✓ TemporalFeatureEngineer fitted")
print(f"  Training samples: {len(train_data)}")
print(f"  Features in training: {list(train_data.columns)}")
```

```
✓ TemporalFeatureEngineer fitted
  Training samples: 1096
  Features in training: ['price', 'inventory', 'production', 'consumption',
'zscore_wrong', 'zscore_correct']
```

Part 3 : Price-Based Features

Price-based features capture market dynamics:

- **Returns**: Price changes over various horizons
- **Volatility**: Risk and uncertainty measures
- **Z-scores**: Relative price levels
- **Momentum**: Trending behavior

3.1 Returns at Multiple Horizons

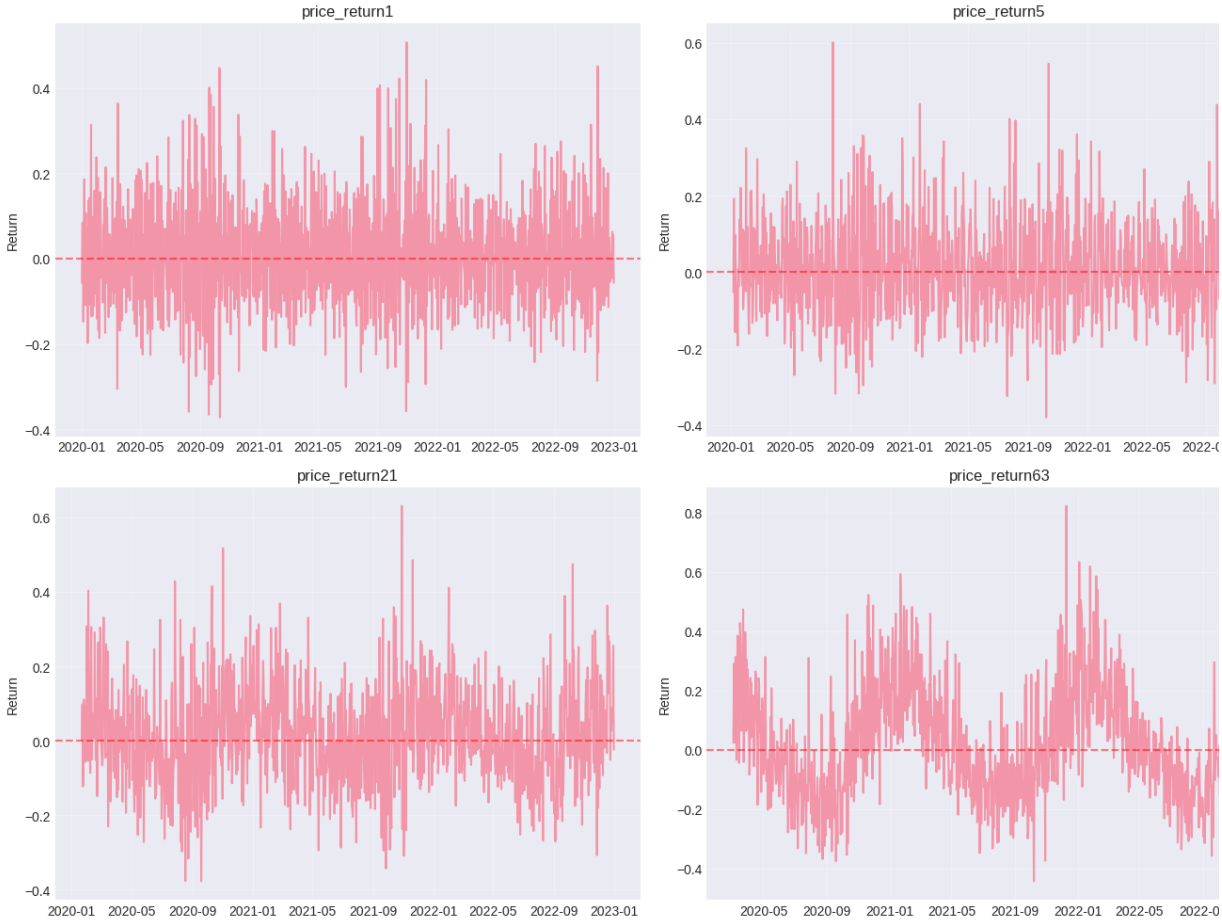
In [6]:
```
# Add return features
features_df = train_data.copy()

# Returns over different periods
periods = [1, 5, 21, 63]  # 1d, 1w, 1m, 3m
features_df = engineer.add_returns(
    features_df,
    columns=['price'],
    periods=periods,
    log_return=False
)

print("✓ Return features added")
print(f"\nNew columns: {[c for c in features_df.columns if 'return' in c]
print(f"\nSample values:")
features_df[['price'] + [c for c in features_df.columns if 'return' in c]
```

```
✓ Return features added

New columns: ['price_return1', 'price_return5', 'price_return21', 'price_retu

Sample values:
```

Out[6]:

	price	price_return 1	price_return 5	price_return 21
2022-12-27	63.9429	0.0056	-0.0646	0.0633
2022-12-28	67.9524	0.0627	0.0521	0.0799
2022-12-29	64.7594	-0.0470	0.0398	0.0600
2022-12-30	68.3485	0.0554	0.0887	0.2548
2022-12-31	64.4674	-0.0568	0.0139	-0.0238

In [7]:
```
# Visualize returns
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.ravel()

return_cols = [c for c in features_df.columns if 'return' in c]
for idx, col in enumerate(return_cols[:4]):
    axes[idx].plot(features_df.index, features_df[col], alpha=0.7)
    axes[idx].axhline(0, color='red', linestyle='--', alpha=0.5)
    axes[idx].set_title(col)
    axes[idx].set_ylabel('Return')
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Calculate correlations
corr_matrix = features_df[return_cols].corr()
print("\nReturn Correlations:")
print(corr_matrix)
```




```
Return Correlations:
               price_return1  price_return5  price_return21  price_return63
price_return1          1.0000         0.4603          0.4408          0.3680
price_return5          0.4603         1.0000          0.4923          0.3996
price_return21         0.4408         0.4923          1.0000          0.5424
price_return63         0.3680         0.3996          0.5424          1.0000
```

3.2 Volatility Measures

Volatility captures uncertainty and risk. High volatility periods often precede mean reversion or re changes.

In [8]:

```
# Add volatility features
vol_windows = [5, 21, 63]  # 1w, 1m, 3m
features_df = engineer.add_volatility(
    features_df,
    columns=['price'],
    windows=vol_windows,
    method='std'
)

print("✓ Volatility features added")
vol_cols = [c for c in features_df.columns if 'vol' in c]
print(f"Volatility columns: {vol_cols}")

# Plot
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Price
axes[0].plot(features_df.index, features_df['price'], label='Price', colo
axes[0].set_ylabel('Price ($)')
axes[0].set_title('Commodity Price')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Volatility
for col in vol_cols:
    axes[1].plot(features_df.index, features_df[col], label=col, alpha=0.
axes[1].set_ylabel('Annualized Volatility')
axes[1].set_xlabel('Date')
axes[1].set_title('Rolling Volatility (Multiple Windows)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
✓ Volatility features added
Volatility columns: ['price_vol5', 'price_vol21', 'price_vol63']
```

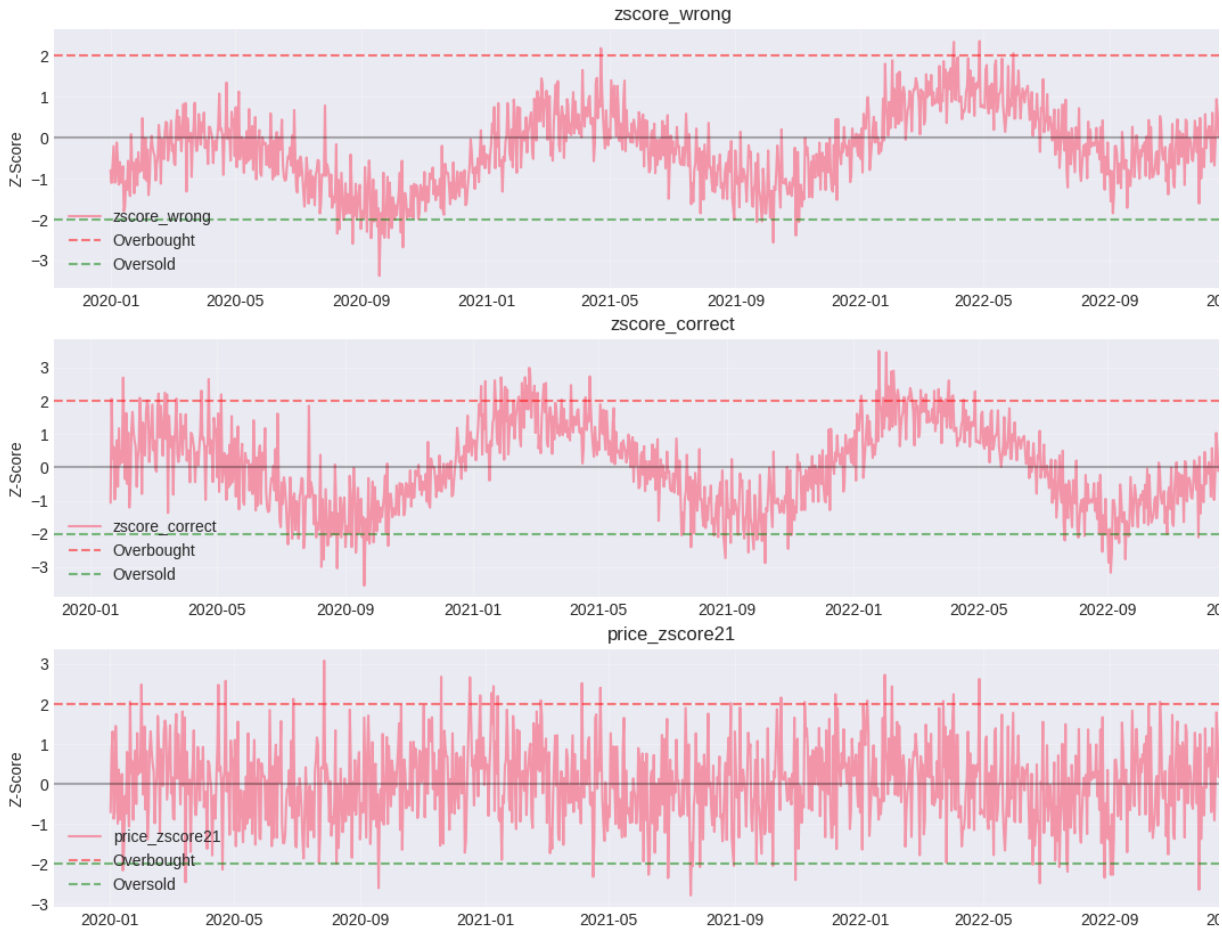
3.3 Z-Scores and Momentum

Z-scores measure how far price has deviated from its historical mean, normalized by volatility. Tl
crucial for mean reversion strategies.

In [9]:
```
# Add z-score features
zscore_windows = [21, 63, 252]  # 1m, 3m, 1y
features_df = engineer.add_zscore(
    features_df,
    columns=['price'],
    windows=zscore_windows
)

print("✓ Z-score features added")
zscore_cols = [c for c in features_df.columns if 'zscore' in c]
print(f"Z-score columns: {zscore_cols}")

# Plot z-scores
fig, axes = plt.subplots(3, 1, figsize=(14, 10))

for idx, col in enumerate(zscore_cols):
    axes[idx].plot(features_df.index, features_df[col], label=col, alpha=
    axes[idx].axhline(0, color='black', linestyle='-', alpha=0.3)
    axes[idx].axhline(2, color='red', linestyle='--', alpha=0.5, label='O
    axes[idx].axhline(-2, color='green', linestyle='--', alpha=0.5, label
    axes[idx].set_ylabel('Z-Score')
    axes[idx].set_title(col)
    axes[idx].legend()
    axes[idx].grid(True, alpha=0.3)

axes[-1].set_xlabel('Date')
plt.tight_layout()
plt.show()

# Statistics
print("\nZ-Score Statistics:")
print(features_df[zscore_cols].describe())
```

```
✓ Z-score features added
Z-score columns: ['zscore_wrong', 'zscore_correct', 'price_zscore21',
'price_zscore63', 'price_zscore252']
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[9], line 17
     14 fig, axes = plt.subplots(3, 1, figsize=(14, 10))
     16 for idx, col in enumerate(zscore_cols):
---> 17     axes[idx].plot(features_df.index, features_df[col], label=col, al
7)
     18     axes[idx].axhline(0, color='black', linestyle='-', alpha=0.3)
     19     axes[idx].axhline(2, color='red', linestyle='--', alpha=0.5, labe
rbought')

IndexError: index 3 is out of bounds for axis 0 with size 3
```



In [10]:
```
# Add rolling statistics for momentum
features_df = engineer.add_rolling_stats(
    features_df,
    columns=['price'],
    windows=[5, 21],
    stats=['mean', 'min', 'max']
)

print("✓ Rolling statistics added")
roll_cols = [c for c in features_df.columns if 'roll' in c]
print(f"Rolling stat columns: {roll_cols}")
```

```
✓ Rolling statistics added
Rolling stat columns: ['price_roll5_mean', 'price_roll5_min', 'price_roll5_ma
'price_roll21_mean', 'price_roll21_min', 'price_roll21_max']
```

Part 4 : Fundamental Features

Fundamental features encode economic theories about what drives commodity prices:

- **Inventory levels**: Storage theory (Working, Kaldor)
- **Supply-demand balance**: Market clearing
- **Cost curves**: Marginal cost of production

4.1 Inventory Features

In [11]:
```
# Add inventory features
# 1. Lagged inventory (storage levels)
features_df = engineer.add_lags(
    features_df,
```

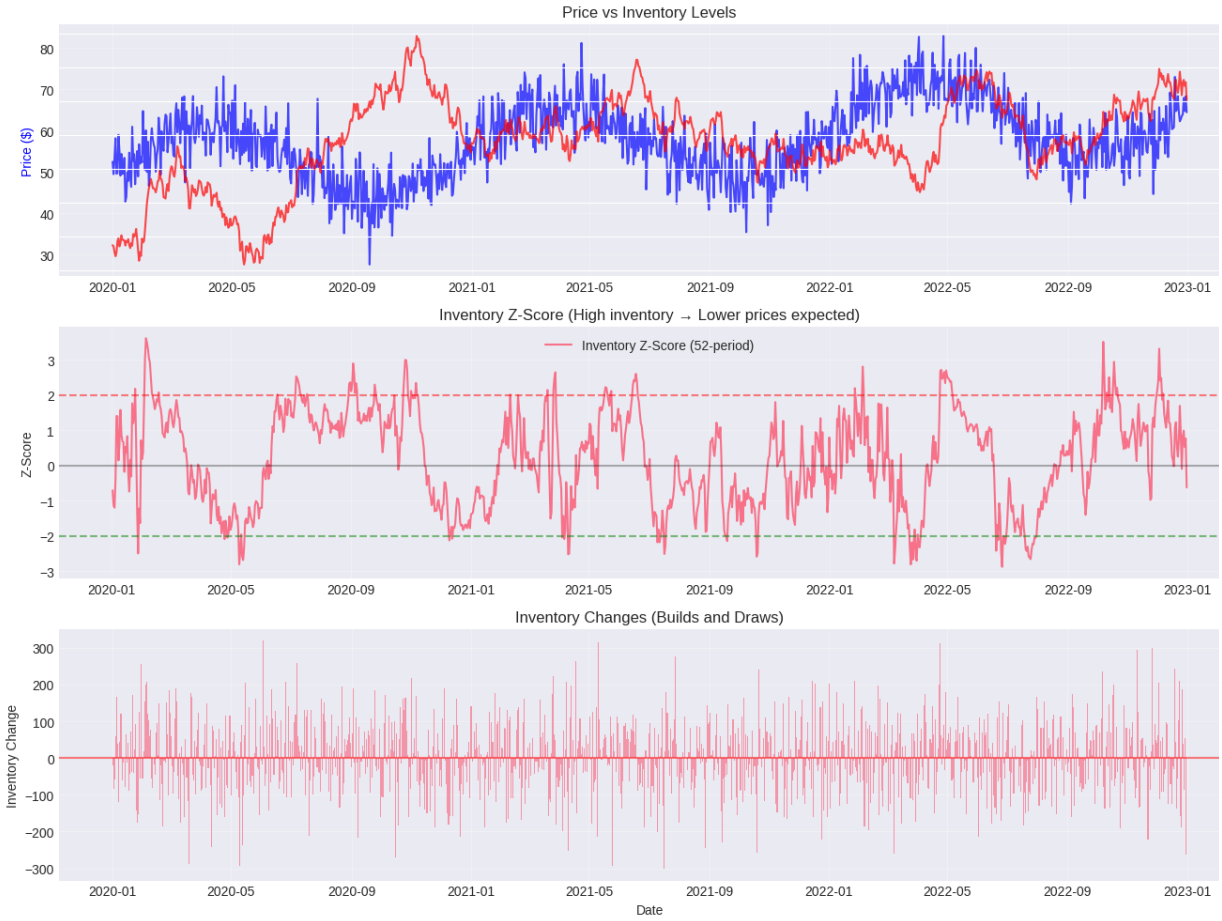
```
    columns=['inventory'],
    lags=[1, 5, 10]
)

# 2. Inventory z-scores (deviation from normal)
features_df = engineer.add_zscore(
    features_df,
    columns=['inventory'],
    windows=[12, 52]  # ~2 weeks, ~2 months (assuming weekly data)
)

# 3. Rolling inventory statistics
features_df = engineer.add_rolling_stats(
    features_df,
    columns=['inventory'],
    windows=[4, 12],
    stats=['mean', 'std']
)

# 4. Inventory changes
features_df['inventory_change'] = features_df['inventory'].diff()
features_df['inventory_change_pct'] = features_df['inventory'].pct_change

print("✓ Inventory features created")
inventory_cols = [c for c in features_df.columns if 'inventory' in c]
print(f"\nInventory features ({len(inventory_cols)}):")
for col in inventory_cols[:10]:  # Show first 10
    print(f"  - {col}")
```

```
✓ Inventory features created

Inventory features (12):
  - inventory
  - inventory_lag1
  - inventory_lag5
  - inventory_lag10
  - inventory_zscore12
  - inventory_zscore52
  - inventory_roll4_mean
  - inventory_roll4_std
  - inventory_roll12_mean
  - inventory_roll12_std
```

In [12]:

```
# Visualize inventory-price relationship
fig, axes = plt.subplots(3, 1, figsize=(14, 10))

# Price and inventory levels
ax1 = axes[0]
ax2 = ax1.twinx()
ax1.plot(features_df.index, features_df['price'], 'b-', label='Price', al
ax2.plot(features_df.index, features_df['inventory'], 'r-', label='Invent
ax1.set_ylabel('Price ($)', color='b')
ax2.set_ylabel('Inventory', color='r')
ax1.set_title('Price vs Inventory Levels')
ax1.grid(True, alpha=0.3)

# Inventory z-score
axes[1].plot(features_df.index, features_df['inventory_zscore52'], label=
axes[1].axhline(0, color='black', linestyle='-', alpha=0.3)
axes[1].axhline(2, color='red', linestyle='--', alpha=0.5)
```

```
axes[1].axhline(-2, color='green', linestyle='--', alpha=0.5)
axes[1].set_ylabel('Z-Score')
axes[1].set_title('Inventory Z-Score (High inventory → Lower prices expec
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Inventory changes
axes[2].bar(features_df.index, features_df['inventory_change'], alpha=0.7
axes[2].axhline(0, color='red', linestyle='-', alpha=0.5)
axes[2].set_ylabel('Inventory Change')
axes[2].set_xlabel('Date')
axes[2].set_title('Inventory Changes (Builds and Draws)')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Correlation analysis
print("\nInventory-Price Correlation Analysis:")
print(f"Price vs Inventory: {features_df['price'].corr(features_df['inven
print(f"Price vs Inventory Change: {features_df['price'].corr(features_df
print(f"Price vs Inventory Z-Score: {features_df['price'].corr(features_d
```




```
Inventory-Price Correlation Analysis:
Price vs Inventory: -0.012
Price vs Inventory Change: -0.009
Price vs Inventory Z-Score: -0.088
```

4.2 Supply-Demand Balance Features

The fundamental equation: **Price = f(Demand - Supply)**

When demand exceeds supply, prices rise. When supply exceeds demand, prices fall.

In [13]:
```
# Create supply-demand balance features
# 1. Raw balance
features_df['supply_demand_balance'] = features_df['production'] - featur

# 2. Rolling balance statistics
features_df = engineer.add_rolling_stats(
    features_df,
    columns=['supply_demand_balance'],
    windows=[4, 12],
    stats=['mean', 'sum']
)

# 3. Cumulative balance (proxy for inventory change)
features_df['cumulative_balance'] = features_df['supply_demand_balance'].

# 4. Balance z-score
features_df = engineer.add_zscore(
    features_df,
    columns=['supply_demand_balance'],
    windows=[52]
)

print("✓ Supply-demand features created")

# Visualize
fig, axes = plt.subplots(3, 1, figsize=(14, 10))

# Supply vs Demand
axes[0].plot(features_df.index, features_df['production'], label='Product
axes[0].plot(features_df.index, features_df['consumption'], label='Consum
axes[0].set_ylabel('Quantity')
axes[0].set_title('Supply vs Demand')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Balance
axes[1].bar(features_df.index, features_df['supply_demand_balance'],
           color=['red' if x < 0 else 'green' for x in features_df['suppl
           alpha=0.6, width=1)
axes[1].axhline(0, color='black', linestyle='-', alpha=0.5)
axes[1].set_ylabel('Balance')
axes[1].set_title('Supply-Demand Balance (Positive = Surplus, Negative =
axes[1].grid(True, alpha=0.3)

# Price overlay
ax_price = axes[2]
ax_balance = ax_price.twinx()
ax_price.plot(features_df.index, features_df['price'], 'b-', label='Price
ax_balance.plot(features_df.index, features_df['supply_demand_balance_rol
                'r-', label='12-Period Rolling Balance', alpha=0.7)
ax_price.set_ylabel('Price ($)', color='b')
ax_balance.set_ylabel('Rolling Balance', color='r')
ax_price.set_xlabel('Date')
```

```
ax_price.set_title('Price vs Supply-Demand Balance')
ax_price.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nSupply-Demand Correlation:")
print(f"Price vs Balance: {features_df['price'].corr(features_df['supply_
print(f"Price vs Rolling Balance: {features_df['price'].corr(features_df[
```

```
✓ Supply-demand features created
```

```
Supply-Demand Correlation:
Price vs Balance: -0.014
Price vs Rolling Balance: -0.056
```

4.3 Cost Curve Features

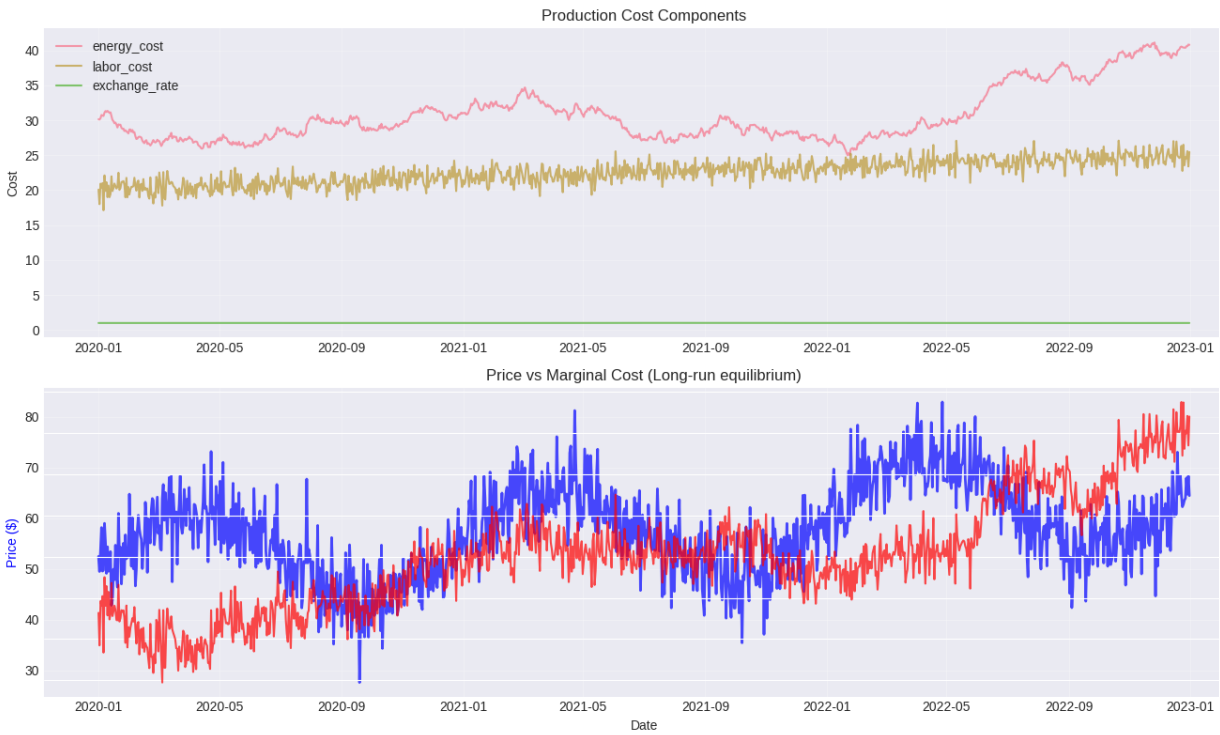
In competitive markets, price gravitates toward the marginal cost of production. We can proxy thi

- Input costs (energy, labor)
- Exchange rates
- Technology indices

In [14]:
```
# Simulate cost curve proxies
np.random.seed(42)
features_df['energy_cost'] = 30 + np.random.normal(0, 3, len(features_df)
features_df['labor_cost'] = 20 + np.linspace(0, 5, len(features_df)) + np
features_df['exchange_rate'] = 1.0 + np.random.normal(0, 0.05, len(featur

# Create composite marginal cost
# Simple weighted average of normalized costs
from sklearn.preprocessing import StandardScaler
```

```
cost_components = ['energy_cost', 'labor_cost', 'exchange_rate']
scaler = StandardScaler()
scaled_costs = scaler.fit_transform(features_df[cost_components])

# Weighted marginal cost (energy has higher weight)
weights = np.array([0.5, 0.3, 0.2])
features_df['marginal_cost_index'] = scaled_costs @ weights

# Add rolling cost features
features_df = engineer.add_rolling_stats(
    features_df,
    columns=['marginal_cost_index'],
    windows=[21],
    stats=['mean', 'std']
)

print("✓ Cost curve features created")

# Visualize
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Cost components
for comp in cost_components:
    axes[0].plot(features_df.index, features_df[comp], label=comp, alpha=
axes[0].set_ylabel('Cost')
axes[0].set_title('Production Cost Components')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Price vs marginal cost
ax1 = axes[1]
ax2 = ax1.twinx()
ax1.plot(features_df.index, features_df['price'], 'b-', label='Price', al
ax2.plot(features_df.index, features_df['marginal_cost_index'], 'r-', lab
ax1.set_ylabel('Price ($)', color='b')
ax2.set_ylabel('Cost Index', color='r')
ax1.set_xlabel('Date')
ax1.set_title('Price vs Marginal Cost (Long-run equilibrium)')
ax1.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nCost-Price Correlation:")
print(f"Price vs Marginal Cost Index: {features_df['price'].corr(features
```

```
✓ Cost curve features created
```




```
Cost-Price Correlation:
Price vs Marginal Cost Index: 0.202
```

Part 5 : Seasonal and Interaction Features

5.1 Calendar Effects

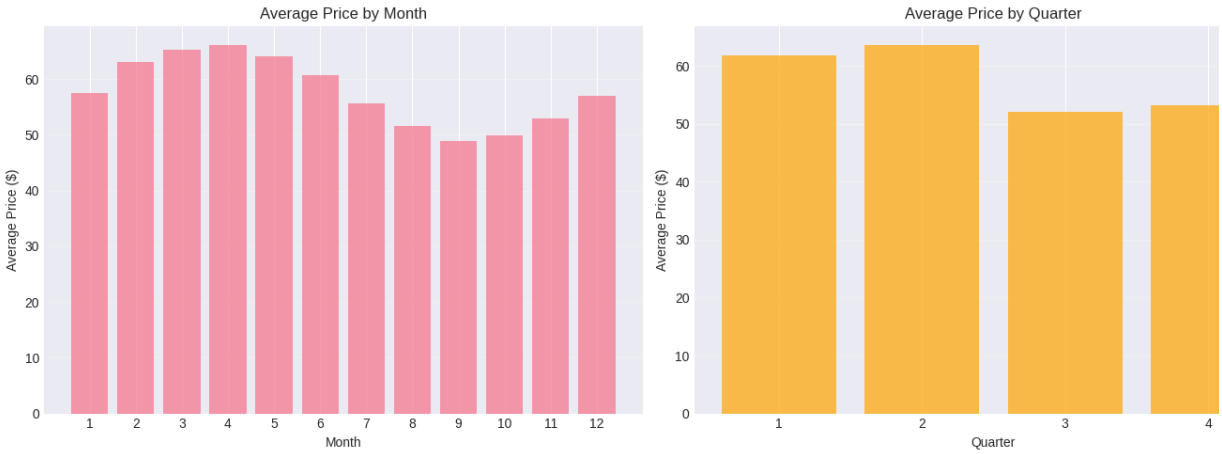
Commodities exhibit seasonal patterns due to:

- Weather (heating oil in winter, cooling demand in summer)
- Crop cycles (agricultural commodities)
- Industrial production patterns
- Holiday effects

In [15]:
```
# Add seasonal features
features_df = engineer.add_seasonal(
    features_df,
    include=['month', 'quarter', 'dayofweek', 'is_month_end', 'is_quarter
)

print("✓ Seasonal features added")
seasonal_cols = ['month', 'quarter', 'dayofweek', 'is_month_end', 'is_qua
print(f"\nSeasonal columns: {seasonal_cols}")

# Analyze seasonality
monthly_avg = features_df.groupby('month')['price'].mean()
quarterly_avg = features_df.groupby('quarter')['price'].mean()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Monthly seasonality
axes[0].bar(monthly_avg.index, monthly_avg.values, alpha=0.7)
axes[0].set_xlabel('Month')
axes[0].set_ylabel('Average Price ($)')
axes[0].set_title('Average Price by Month')
axes[0].set_xticks(range(1, 13))
```

```
axes[0].grid(True, alpha=0.3, axis='y')

# Quarterly seasonality
axes[1].bar(quarterly_avg.index, quarterly_avg.values, alpha=0.7, color='
axes[1].set_xlabel('Quarter')
axes[1].set_ylabel('Average Price ($)')
axes[1].set_title('Average Price by Quarter')
axes[1].set_xticks(range(1, 5))
axes[1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# Statistical test for seasonality
from scipy.stats import f_oneway

monthly_groups = [features_df[features_df['month'] == m]['price'].dropna(
f_stat, p_value = f_oneway(*monthly_groups)

print(f"\nANOVA Test for Monthly Seasonality:")
print(f"F-statistic: {f_stat:.2f}")
print(f"P-value: {p_value:.4f}")
if p_value < 0.05:
    print("✓ Statistically significant seasonal pattern detected")
else:
    print("✗ No significant seasonal pattern")
```

```
✓ Seasonal features added

Seasonal columns: ['month', 'quarter', 'dayofweek', 'is_month_end',
'is_quarter_end']
```




```
ANOVA Test for Monthly Seasonality:
F-statistic: 78.01
P-value: 0.0000
✓ Statistically significant seasonal pattern detected
```

5.2 Cross-Commodity Spreads

Commodities don't trade in isolation. Related commodities can provide signals:

- Crude oil vs refined products (crack spread)
- Corn vs ethanol
- Natural gas vs power

In [16]:

```
# Simulate related commodity
np.random.seed(42)
features_df['related_commodity'] = features_df['price'] * 1.2 + np.random

# Create spread features
features_df['commodity_spread'] = features_df['price'] - features_df['rel
features_df['commodity_ratio'] = features_df['price'] / features_df['rela

# Rolling spread statistics
features_df = engineer.add_rolling_stats(
    features_df,
    columns=['commodity_spread'],
    windows=[21],
    stats=['mean', 'std']
)

# Spread z-score
features_df = engineer.add_zscore(
    features_df,
    columns=['commodity_spread'],
    windows=[63]
)

print("✓ Cross-commodity spread features created")

# Visualize
fig, axes = plt.subplots(3, 1, figsize=(14, 10))

# Prices
axes[0].plot(features_df.index, features_df['price'], label='Primary Comm
axes[0].plot(features_df.index, features_df['related_commodity'], label='
axes[0].set_ylabel('Price ($)')
axes[0].set_title('Primary vs Related Commodity')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Spread
axes[1].plot(features_df.index, features_df['commodity_spread'], alpha=0.
axes[1].axhline(features_df['commodity_spread'].mean(), color='red', line
axes[1].set_ylabel('Spread ($)')
axes[1].set_title('Commodity Spread (Primary - Related)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Spread z-score
axes[2].plot(features_df.index, features_df['commodity_spread_zscore63'],
axes[2].axhline(0, color='black', linestyle='-', alpha=0.3)
axes[2].axhline(2, color='red', linestyle='--', alpha=0.5, label='Expensi
axes[2].axhline(-2, color='green', linestyle='--', alpha=0.5, label='Chea
axes[2].set_ylabel('Z-Score')
axes[2].set_xlabel('Date')
axes[2].set_title('Spread Z-Score (Mean Reversion Signal)')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nSpread Statistics:")
print(f"Mean spread: ${features_df['commodity_spread'].mean():.2f}")
```

```
print(f"Spread volatility: ${features_df['commodity_spread'].std():.2f}")
print(f"Spread correlation with price: {features_df['price'].corr(feature
```

```
✓ Cross-commodity spread features created
```

```
Spread Statistics:
Mean spread: $-11.63
Spread volatility: $4.19
Spread correlation with price: -0.817
```

5.3 Interaction Features

Sometimes the combination of features is more predictive than individual features:

- High inventory + low demand = very bearish
- Low inventory + high volatility = explosive upside

In [17]:
```
# Create interaction features
interactions = [
    ('inventory_zscore52', 'supply_demand_balance'),
    ('price_vol21', 'inventory_zscore52'),
]

features_df = engineer.add_interactions(
    features_df,
    interactions=interactions,
    operation='multiply'
)

print("✓ Interaction features created")
interaction_cols = [c for c in features_df.columns if '_x_' in c]
print(f"Interaction features: {interaction_cols}")
```

```
# Analyze predictive power
print("\nInteraction Feature Correlations with Price:")
for col in interaction_cols:
    corr = features_df['price'].corr(features_df[col])
    print(f"{col}: {corr:.3f}")
```

```
✓ Interaction features created
Interaction features: ['inventory_zscore52_x_supply_demand_balance',
'price_vol21_x_inventory_zscore52']

Interaction Feature Correlations with Price:
inventory_zscore52_x_supply_demand_balance: -0.026
price_vol21_x_inventory_zscore52: -0.138
```

5.4 Relative Strength

Relative strength measures how one commodity performs versus a benchmark.

In [18]:
```
# Add relative strength features
features_df = engineer.add_relative_strength(
    features_df,
    column='price',
    benchmark_column='related_commodity',
    windows=[21, 63]
)

print("✓ Relative strength features added")
rs_cols = [c for c in features_df.columns if '_rs_' in c]
print(f"Relative strength columns: {rs_cols}")

# Plot
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Relative strength
for col in rs_cols:
    axes[0].plot(features_df.index, features_df[col], label=col, alpha=0.
axes[0].axhline(0, color='black', linestyle='-', alpha=0.3)
axes[0].set_ylabel('Relative Strength')
axes[0].set_title('Relative Strength vs Benchmark')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Cumulative relative performance
price_ret = features_df['price'].pct_change().fillna(0).cumsum()
bench_ret = features_df['related_commodity'].pct_change().fillna(0).cumsu
axes[1].plot(features_df.index, price_ret, label='Primary Commodity', alp
axes[1].plot(features_df.index, bench_ret, label='Related Commodity', alp
axes[1].set_ylabel('Cumulative Return')
axes[1].set_xlabel('Date')
axes[1].set_title('Cumulative Performance Comparison')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
✓ Relative strength features added
Relative strength columns: ['price_rs_related_commodity_21',
'price_rs_related_commodity_63']
```

Part 6 : Feature Metadata and Inspection

The `TemporalFeatureEngineer` tracks metadata for all created features. This is crucial for:

- Understanding feature lineage
- Identifying maximum lookback period
- Documenting model inputs

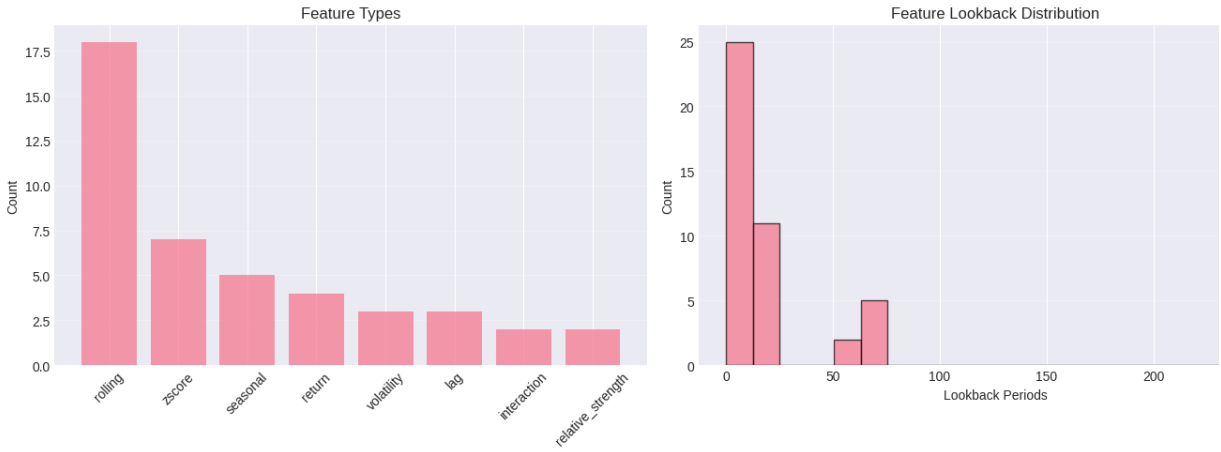
In [19]:
```
# Get feature metadata
feature_info = engineer.get_feature_info()

print(f"Total registered features: {len(feature_info)}")
print(f"\nFeature metadata sample:")
print(feature_info.head(15))
```

```
Total registered features: 44

Feature metadata sample:
                 name source        type  lookback  \
0       price_return1  price      return         1
1       price_return5  price      return         5
2      price_return21  price      return        21
3      price_return63  price      return        63
4          price_vol5  price  volatility         5
5         price_vol21  price  volatility        21
6         price_vol63  price  volatility        63
7      price_zscore21  price      zscore        21
8      price_zscore63  price      zscore        63
9     price_zscore252  price      zscore       252
10   price_roll5_mean  price     rolling         5
11    price_roll5_min  price     rolling         5
12    price_roll5_max  price     rolling         5
13  price_roll21_mean  price     rolling        21
14   price_roll21_min  price     rolling        21

                                 description
0            1-period simple return of price
1            5-period simple return of price
2           21-period simple return of price
3           63-period simple return of price
4    5-period annualized volatility of price
5   21-period annualized volatility of price
6   63-period annualized volatility of price
7                 21-period z-score of price
8                 63-period z-score of price
9                252-period z-score of price
10            5-period rolling mean of price
11             5-period rolling min of price
12             5-period rolling max of price
13           21-period rolling mean of price
14            21-period rolling min of price
```

In [20]:

```
# Analyze feature types
feature_type_counts = feature_info['type'].value_counts()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Feature type distribution
axes[0].bar(feature_type_counts.index, feature_type_counts.values, alpha=
axes[0].set_ylabel('Count')
axes[0].set_title('Feature Types')
axes[0].tick_params(axis='x', rotation=45)
axes[0].grid(True, alpha=0.3, axis='y')

# Lookback period distribution
axes[1].hist(feature_info['lookback'], bins=20, alpha=0.7, edgecolor='bla
axes[1].set_xlabel('Lookback Periods')
axes[1].set_ylabel('Count')
axes[1].set_title('Feature Lookback Distribution')
axes[1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print(f"\nFeature Type Distribution:")
```

```
print(feature_type_counts)
print(f"\nMaximum lookback period: {engineer.get_max_lookback()} periods"
```




```
Feature Type Distribution:
type
rolling              18
zscore                7
seasonal              5
return                4
volatility            3
lag                   3
interaction           2
relative_strength     2
Name: count, dtype: int64

Maximum lookback period: 252 periods
```

Part 7 : The Complete Feature Set

Let's create a comprehensive feature set using the convenience function `create_commodity_features()`.

In [21]:
```
# Start fresh with original data
commodity_data = df.loc[:train_end].copy()

# Use the convenience function
full_features = create_commodity_features(
    commodity_data,
    price_column='price',
    inventory_column='inventory',
    production_column='production',
    consumption_column='consumption'
)

print(f"✓ Complete feature set created")
print(f"\nOriginal columns: {len(commodity_data.columns)}")
print(f"Feature-engineered columns: {len(full_features.columns)}")
print(f"New features added: {len(full_features.columns) - len(commodity_d

print(f"\nFeature columns:")
for i, col in enumerate(full_features.columns, 1):
    print(f"{i:3d}. {col}")
```

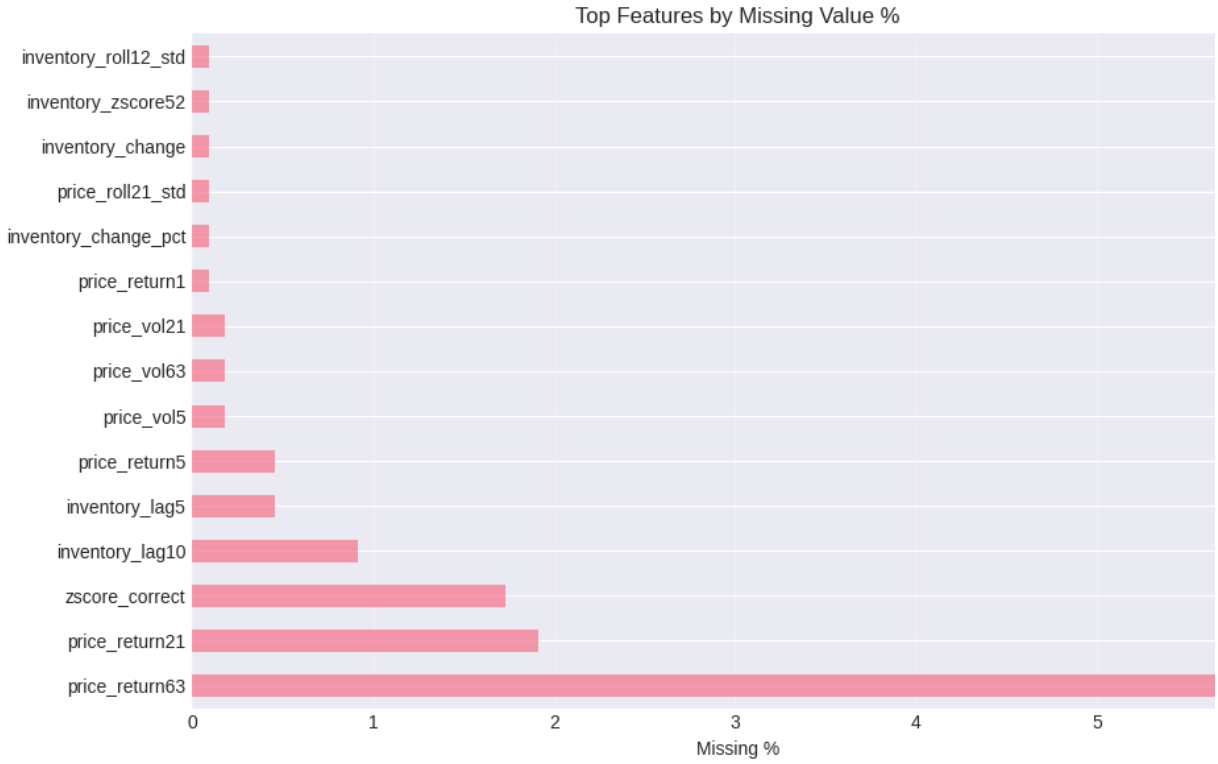
```
✓ Complete feature set created

Original columns: 6
Feature-engineered columns: 42
New features added: 36

Feature columns:
  1. price
  2. inventory
  3. production
  4. consumption
  5. zscore_wrong
  6. zscore_correct
  7. price_return1
  8. price_return5
  9. price_return21
 10. price_return63
 11. price_vol5
 12. price_vol21
 13. price_vol63
 14. price_zscore21
 15. price_zscore63
 16. price_zscore252
 17. price_roll5_mean
 18. price_roll5_std
 19. price_roll5_min
 20. price_roll5_max
 21. price_roll21_mean
 22. price_roll21_std
 23. price_roll21_min
 24. price_roll21_max
 25. inventory_lag1
 26. inventory_lag5
 27. inventory_lag10
 28. inventory_roll4_mean
 29. inventory_roll4_std
 30. inventory_roll12_mean
 31. inventory_roll12_std
 32. inventory_zscore12
 33. inventory_zscore52
 34. inventory_change
 35. inventory_change_pct
 36. supply_demand_balance
 37. supply_demand_balance_roll4_mean
 38. supply_demand_balance_roll4_sum
 39. supply_demand_balance_roll12_mean
 40. supply_demand_balance_roll12_sum
 41. month
 42. quarter
```

In [22]:

```
# Check for missing values
missing_pct = (full_features.isnull().sum() / len(full_features) * 100).s
missing_pct = missing_pct[missing_pct > 0]

if len(missing_pct) > 0:
    print("Features with missing values:")
    print(missing_pct.head(10))

    fig, ax = plt.subplots(figsize=(10, 6))
    missing_pct.head(15).plot(kind='barh', ax=ax, alpha=0.7)
```

```
    ax.set_xlabel('Missing %')
    ax.set_title('Top Features by Missing Value %')
    ax.grid(True, alpha=0.3, axis='x')
    plt.tight_layout()
    plt.show()
else:
    print("✓ No missing values in feature set")
```

```
Features with missing values:
price_return63     5.7482
price_return21     1.9161
zscore_correct     1.7336
inventory_lag10    0.9124
inventory_lag5     0.4562
price_return5      0.4562
price_vol5         0.1825
price_vol63        0.1825
price_vol21        0.1825
price_return1      0.0912
dtype: float64
```



Part 8 : Feature Correlation Analysis

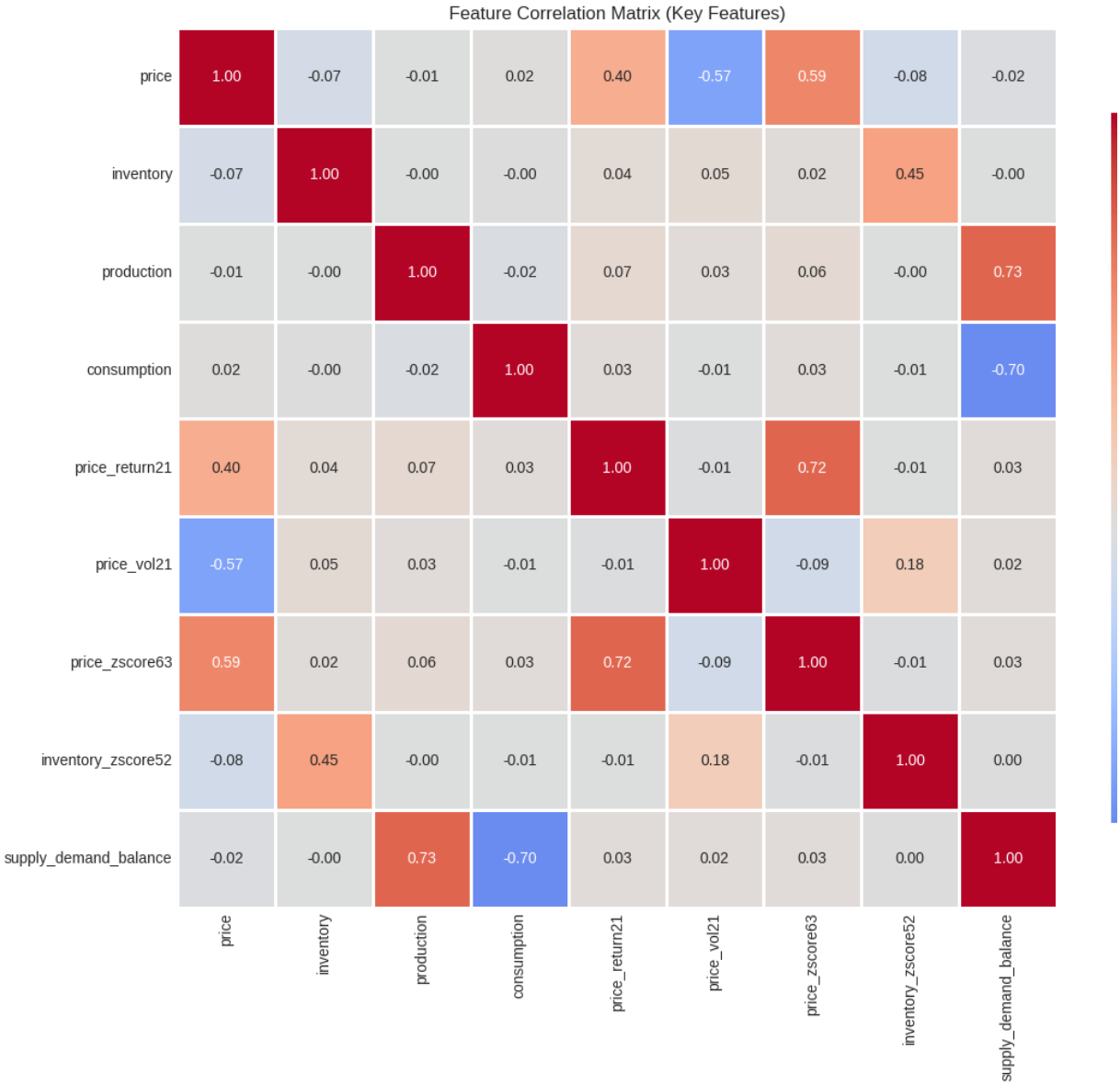
High correlations between features (multicollinearity) can cause:

- Unstable coefficient estimates
- Poor out-of-sample performance
- Difficulty interpreting feature importance

In [23]:
```
# Select numeric features for correlation analysis
numeric_features = full_features.select_dtypes(include=[np.number]).dropn

# Calculate correlation matrix
corr_matrix = numeric_features.corr()

# Find highly correlated pairs
```

```python
high_corr_pairs = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        if abs(corr_matrix.iloc[i, j]) > 0.95:
            high_corr_pairs.append((
                corr_matrix.columns[i],
                corr_matrix.columns[j],
                corr_matrix.iloc[i, j]
            ))

print(f"Highly correlated feature pairs (|r| > 0.95):")
for feat1, feat2, corr in high_corr_pairs[:10]:
    print(f"  {feat1} ↔ {feat2}: {corr:.3f}")

# Visualize correlation heatmap (subset)
key_features = ['price', 'inventory', 'production', 'consumption',
                'price_return21', 'price_vol21', 'price_zscore63',
                'inventory_zscore52', 'supply_demand_balance']
key_features = [f for f in key_features if f in corr_matrix.columns]

fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(corr_matrix.loc[key_features, key_features],
            annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8}, ax=ax)
ax.set_title('Feature Correlation Matrix (Key Features)')
plt.tight_layout()
plt.show()
```

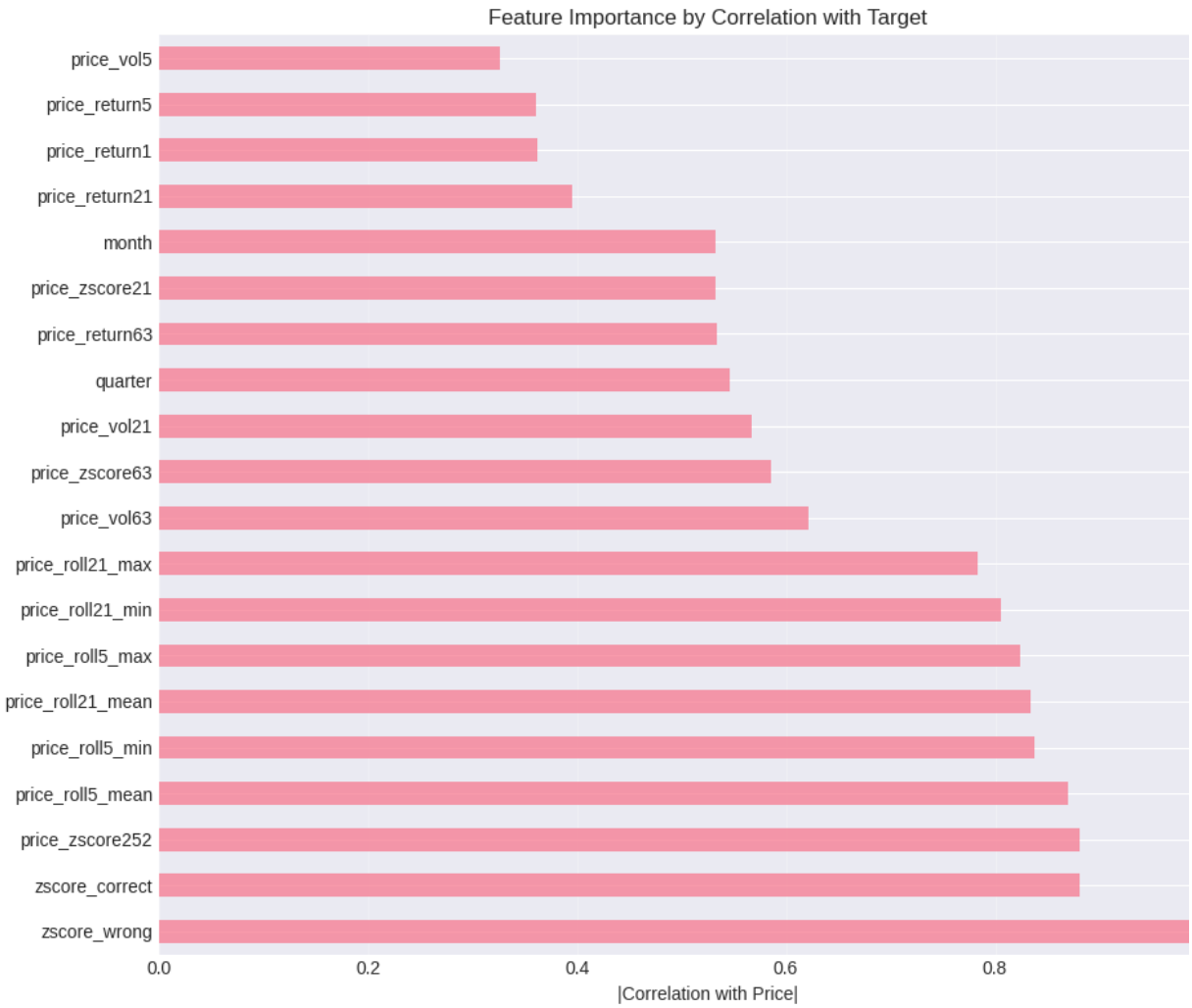
```
Highly correlated feature pairs (|r| > 0.95):
  price ↔ zscore_wrong: 1.000
  inventory ↔ inventory_lag1: 0.987
  inventory ↔ inventory_roll4_mean: 0.989
  inventory ↔ inventory_roll12_mean: 0.962
  zscore_correct ↔ price_zscore252: 1.000
  price_roll5_mean ↔ price_roll5_min: 0.960
  price_roll5_mean ↔ price_roll5_max: 0.952
  price_roll5_mean ↔ price_roll21_mean: 0.963
  price_roll21_mean ↔ price_roll21_min: 0.964
  inventory_lag1 ↔ inventory_lag5: 0.953
```



In [24]:

```
# Calculate feature importance via correlation with target
target_corrs = corr_matrix['price'].drop('price').abs().sort_values(ascen

print("\nTop 20 Features by Correlation with Price:")
print(target_corrs.head(20))

# Plot
fig, ax = plt.subplots(figsize=(10, 8))
target_corrs.head(20).plot(kind='barh', ax=ax, alpha=0.7)
ax.set_xlabel('|Correlation with Price|')
ax.set_title('Feature Importance by Correlation with Target')
ax.grid(True, alpha=0.3, axis='x')
plt.tight_layout()
plt.show()
```

```
Top 20 Features by Correlation with Price:
zscore_wrong         1.0000
zscore_correct       0.8807
price_zscore252      0.8807
price_roll5_mean     0.8693
price_roll5_min      0.8371
price_roll21_mean    0.8338
price_roll5_max      0.8244
price_roll21_min     0.8050
price_roll21_max     0.7836
price_vol63          0.6220
price_zscore63       0.5859
price_vol21          0.5667
quarter              0.5464
price_return63       0.5337
price_zscore21       0.5327
month                0.5326
price_return21       0.3955
price_return1        0.3624
price_return5        0.3615
price_vol5           0.3259
Name: price, dtype: float64
```



Part 9 : Applying Features to Test Data

A critical test: Can we apply the same transformations to new (test) data?

The engineer must use the same lookback windows and normalization parameters.

In [25]:
```
# Get test data
test_data = df.loc['2023-01-01':].copy()

print(f"Test data shape: {test_data.shape}")
print(f"Test date range: {test_data.index[0].date()} to {test_data.index[

# Apply same feature engineering
test_features = create_commodity_features(
    test_data,
    price_column='price',
    inventory_column='inventory',
    production_column='production',
    consumption_column='consumption'
)

print(f"\nTest features shape: {test_features.shape}")
print(f"✓ Successfully applied feature engineering to test data")

# Verify feature alignment
train_cols = set(full_features.columns)
test_cols = set(test_features.columns)

missing_in_test = train_cols - test_cols
extra_in_test = test_cols - train_cols

if len(missing_in_test) > 0:
    print(f"\n⚠ Features missing in test: {missing_in_test}")
if len(extra_in_test) > 0:
    print(f"\n⚠ Extra features in test: {extra_in_test}")
if len(missing_in_test) == 0 and len(extra_in_test) == 0:
    print(f"\n✓ Train and test features perfectly aligned")
```

```
Test data shape: (365, 6)
Test date range: 2023-01-01 to 2023-12-31
Test features shape: (365, 42)
✓ Successfully applied feature engineering to test data

✓ Train and test features perfectly aligned
```

In [26]:
```
# Visualize feature distribution: train vs test
sample_features = ['price_return21', 'price_vol21', 'inventory_zscore52']
sample_features = [f for f in sample_features if f in full_features.colum

fig, axes = plt.subplots(1, len(sample_features), figsize=(15, 4))
if len(sample_features) == 1:
    axes = [axes]

for idx, feat in enumerate(sample_features):
    axes[idx].hist(full_features[feat].dropna(), bins=30, alpha=0.5, labe
    axes[idx].hist(test_features[feat].dropna(), bins=30, alpha=0.5, labe
    axes[idx].set_xlabel(feat)
    axes[idx].set_ylabel('Density')
    axes[idx].set_title(f'Distribution: {feat}')
    axes[idx].legend()
    axes[idx].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("\nDistribution Statistics (Train vs Test):")
```

```python
for feat in sample_features:
    train_mean = full_features[feat].mean()
    test_mean = test_features[feat].mean()
    train_std = full_features[feat].std()
    test_std = test_features[feat].std()
    print(f"\n{feat}:")
    print(f"  Train: μ={train_mean:.3f}, σ={train_std:.3f}")
    print(f"  Test:  μ={test_mean:.3f}, σ={test_std:.3f}")
```

```
Distribution Statistics (Train vs Test):

price_return21:
  Train: μ=0.013, σ=0.136
  Test:  μ=0.008, σ=0.113

price_vol21:
  Train: μ=1.999, σ=0.506
  Test:  μ=1.822, σ=0.542

inventory_zscore52:
  Train: μ=0.191, σ=1.345
  Test:  μ=0.002, σ=1.416
```

Part 10: Best Practices Summary

✓ Do's

1. **Always use rolling windows** for statistics (mean, std, etc.)
2. **Document feature lineage** with metadata
3. **Use consistent transformations** across train/test
4. **Handle missing values explicitly** (don't assume)
5. **Test on out-of-sample data** to verify no leakage
6. **Normalize lookback periods** (e.g., 21d ≈ 1 month)

✗ Don'ts

1. **Never use `.mean()` or `.std()` on entire dataset**
2. **Don't use future information** in feature calculation
3. **Don't assume data is clean** (always check)
4. **Don't create features that can't be computed in production**
5. **Don't ignore feature correlations** (multicollinearity matters)

Critical Questions to Ask

Before adding any feature:

1. **Is this feature available in real-time?**
   - Can I compute it with only past data?
   - Will it have revisions?
2. **What's the economic intuition?**
   - Why should this predict prices?
   - Is there a fundamental relationship?
3. **How stable is this feature?**
   - Does it work across different time periods?
   - Is it regime-dependent?
4. **What's the feature's half-life?**
   - How quickly does information decay?
   - Should I use a shorter lookback?

The Production Reality Check

Before deploying, ask: Can my feature engineer run at $9:30$ AM on Monday with o available at that moment?

If the answer is no, you have look-ahead bias.

Exercises

1. **Create a "smart money" feature**: Use volume and open interest to identify when informed are active
2. **Build a regime detection feature**: Identify market regimes (trending vs mean-reverting) usi rolling statistics
3. **Engineer a "stress index"**: Combine volatility, inventory levels, and supply-demand balanc single stress measure
4. **Implement Fourier features**: Capture complex seasonal patterns using Fourier basis funct
5. **Create forward curve features**: If you have futures data, build features from the term struc slope and curvature

Key Takeaways

1. **Temporal discipline is non-negotiable** - One instance of look-ahead bias can invalidate y backtest

2. **The `TemporalFeatureEngineer` automates temporal feature creation** while maintai discipline
3. **Price-based features** (returns, volatility, z-scores) capture market dynamics
4. **Fundamental features** (inventory, supply-demand) encode economic theory
5. **Seasonal and interaction features** capture complex patterns and non-linearities
6. **Feature metadata tracking** is essential for documentation and production deployment
7. **Always test features on out-of-sample data** to verify they can be computed in real-time

Next Steps

In **Module 7: Model Validation with Walk-Forward Testing**, we'll learn how to validate fair models using proper time series cross-validation that respects temporal order and prevents data